AWS SAA-C03 Study Guide

Module 3b
Compute, Databases & Performance

EC2 & placement • Containers & serverless • Choosing a database • Caching & CDN • Hybrid network performance • Streaming & analytics • Worked practice questions

1. EC2: Choosing and Placing Compute

1.1 Instance families — read the letter

Instance names encode purpose: family letter + generation + size (c7g.xlarge = compute-optimized, gen 7, Graviton, xlarge). The exam expects you to match a workload to a family:

Family	Optimized for	Typical workload
T (t3, t4g)	Burstable — baseline CPU + credits for spikes	Dev/test, low-traffic web, microservices; 'unlimited' mode avoids throttling
M	General purpose balance	App servers, mid-size DBs, default when unsure
C	Compute (high CPU:RAM)	Batch, encoding, HPC, gaming servers, scientific modelling
R / X / z	Memory (high RAM:CPU)	In-memory DBs, caches, SAP HANA, big-data analytics
I / D / H	Local storage — NVMe IOPS (I) or dense HDD (D/H)	NoSQL/Cassandra, data warehouses, Hadoop
P / G / Inf / Trn	GPU / ML accelerators	Training (P/Trn), inference (Inf/G), graphics rendering (G)
Graviton (suffix g)	ARM CPUs — best price/performance (~20–40% cheaper)	Anything with an ARM-compatible build; the 'reduce cost, same performance' answer

1.2 Placement groups — where instances land physically

Type	Physical layout	Use / limit
Cluster	Packed into one rack/AZ, low-latency 10–100 Gbps between nodes	HPC, MPI, tightly coupled compute. Single AZ (so single point of failure); launch all instances at once
Spread	Each instance on distinct hardware/rack	Small number of CRITICAL instances that must not fail together. Max 7 per AZ (can span AZs)
Partition	Groups of instances on separate rack sets (partitions), failures isolated per partition	Large distributed systems: Hadoop/HDFS, Kafka, Cassandra. Up to 7 partitions/AZ, hundreds of instances

1.3 Networking, metadata, and lifecycle facts

- **ENI** = virtual NIC (private IP, SGs, MAC). Extra ENIs can be moved between instances for failover of a fixed IP. **ENA** = enhanced networking (up to 100+ Gbps, required on modern types). **EFA** = ENA + OS-bypass for MPI/HPC — 'tightly coupled HPC nodes' → cluster placement group + EFA.
- **Instance metadata service** (169.254.169.254) supplies the instance's role credentials, IP, user data. IMDSv2 requires a session token — the mitigation for SSRF credential theft. **User data** = bootstrap script, runs once at first boot as root.
- **Stop vs terminate vs hibernate:** stop keeps EBS, loses instance store and RAM (public IPv4 changes unless Elastic IP); hibernate saves RAM to the encrypted root EBS for fast resume (long-running in-memory state); terminate deletes root by default.
- **AMIs** are Region-specific (copy to use elsewhere) and the fastest scale-out enabler: bake dependencies into a 'golden AMI' instead of installing on boot. EC2 Image Builder automates AMI pipelines.
- **Elastic IPs** hold a static public IPv4 across stop/start; 5 per Region soft limit. **Spot**: 2-minute interruption warning; use Spot Fleet/ASG mixed-instance policies with capacity-optimized allocation to minimize interruptions.

2. Containers & Serverless Compute

2.1 The compute spectrum

From most control/most ops to least: **EC2** (you own the OS) → **ECS/EKS on EC2** (you own the nodes, AWS schedules containers) → **Fargate** (containers, no nodes) → **Lambda** (functions, no containers). Every 'least operational overhead' question pushes you rightward along this line as far as the workload allows; 'needs OS-level control / specific kernel / GPU passthrough' pushes left.

2.2 ECS, EKS, Fargate, ECR

- **ECS** (Elastic Container Service): AWS-native orchestrator. A **task definition** describes containers (image, CPU/RAM, ports, IAM **task role** = per-task permissions, secrets from Secrets Manager/SSM); a **service** keeps N tasks running behind an ALB (dynamic port mapping lets many tasks share a host). Launch types: EC2 (you manage the cluster instances; can use Spot/RIs) or Fargate.
- **Fargate:** serverless containers — specify CPU/RAM per task, no instances to patch or scale. Costs more per unit than well-packed EC2 but wins on ops. Works for ECS and EKS.
- **EKS** (Elastic Kubernetes Service): managed Kubernetes control plane. Choose only when the requirement names Kubernetes, existing K8s manifests/tooling, or multi-cloud portability. Otherwise ECS is simpler.
- **ECR:** managed private registry with vulnerability scanning and lifecycle policies to prune old images. **App Runner:** give it a container/repo, get a scaling HTTPS service — the simplest web-container option.

2.3 Lambda — limits and patterns you must know

Fact	Value / implication
Max execution time	15 minutes — anything longer → Fargate, Batch, Step Functions splitting work, or EC2
Memory	128 MB – 10 GB; CPU and network scale WITH memory (more memory = faster, sometimes cheaper)
Storage	/tmp up to 10 GB (ephemeral); container images up to 10 GB; deployment zip 250 MB unzipped
Concurrency	1,000 per Region by default (raise via quota). Reserved concurrency = guarantee/cap one function; Provisioned concurrency = pre-warmed instances (kill cold starts, extra cost)
Triggers	Sync: API Gateway, ALB, direct invoke. Async: S3, SNS, EventBridge (retries + DLQ/destinations). Poll-based: SQS, Kinesis, DynamoDB Streams (event source mapping, batch size)
Networking	Runs in AWS-managed VPC; to reach private RDS/ElastiCache attach it to YOUR VPC subnets (needs NAT/endpoint for internet/AWS APIs). RDS connections → use RDS Proxy
Pricing	Per request + GB-seconds; free tier 1M requests/mo. Idle costs nothing
State	Stateless & ephemeral — persist to DynamoDB/S3/EFS (EFS can mount into Lambda for shared large files)

- **Layers** share libraries across functions. **Lambda@Edge / CloudFront Functions** run logic at edge locations (header rewrites, A/B, auth). **SnapStart** cuts Java cold starts.

2.4 API Gateway

Managed front door for APIs. **REST APIs** (full features: caching, usage plans + API keys, request validation/transformation, WAF, canary deployments) vs **HTTP APIs** (cheaper, lower latency, fewer features — the default for simple Lambda proxies) vs **WebSocket APIs** (persistent two-way connections — chat, live dashboards). Endpoint types: edge-optimized (CloudFront-fronted, global clients), Regional, private (only from your VPC via interface endpoint). Auth: IAM, Cognito User Pool authorizer, Lambda authorizer (custom tokens). Default throttle 10,000 rps; integration timeout 29 s (so a 5-minute Lambda behind API Gateway still fails — go async). Caching responses (per-stage) reduces backend hits.

2.5 Other compute you'll see as options

- **Elastic Beanstalk:** PaaS — upload code, it provisions ALB/ASG/RDS and manages deployments; you still see/own the resources. 'Developers want to deploy without managing infra but keep full control' → Beanstalk. **AWS Batch:**

managed batch job queues/scheduling on EC2/Fargate/Spot — 'run thousands of batch jobs, some hours long.' **Amplify:** full-stack web/mobile hosting + backend. **Lightsail:** fixed-price VPS for simple projects. **Outposts:** AWS racks in your datacenter (data residency, ultra-low latency to on-prem systems). **Local Zones:** AWS compute in metro areas for single-digit-ms latency; **Wavelength:** inside 5G carrier networks.

EXAM TIP: Sporadic, event-driven, <15 min → Lambda. Containers with least ops → Fargate. Named Kubernetes → EKS. Hours-long batch → AWS Batch. Need the OS → EC2. Lambda + RDS = RDS Proxy. API >29 s → make it asynchronous (SQS/Step Functions).

3. Choosing a Database

3.1 Relational vs NoSQL — the real decision

Relational (RDS/Aurora) stores rows in tables with a fixed schema, enforces relationships, and gives you joins and ACID transactions — the right fit when data is interconnected and queries are unpredictable (reports, ad-hoc analysis). It scales reads via replicas but writes scale mostly vertically. **NoSQL** (DynamoDB) trades joins and flexible queries for horizontal scale: you design tables around known access patterns, and in return get single-digit-ms latency at any volume with no capacity ceiling. Also weigh **OLTP** (many small reads/writes — transactional apps) vs **OLAP** (few huge scans over history — analytics/BI → Redshift, never an OLTP engine).

CORE CONCEPT — How the exam asks it: Look for: 'complex joins / existing SQL app / strong consistency across tables' → relational. 'Key-value / millisecond latency / massive scale / serverless / flexible schema' → DynamoDB. 'Analytics over petabytes / BI dashboards' → Redshift. 'Sub-millisecond / caching / leaderboard' → in-memory. Then apply managed-ness: Aurora over RDS when performance/scale/HA matter; RDS when a specific engine (Oracle, SQL Server) or exact compatibility is required.

3.2 RDS and Aurora essentials (performance angle)

- **RDS** engines: MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Db2. Storage on EBS (gp3/io1) — IOPS-bound; Storage Auto Scaling grows disk. Multi-AZ for HA, read replicas for reads (see Module 2). **RDS Custom** gives OS/DB access for Oracle/SQL Server customizations. **RDS Proxy** pools connections (Lambda!).
- **Aurora:** up to 5x MySQL / 3x PostgreSQL throughput; 15 replicas, reader endpoint, Global Database, Serverless v2 (scales in 0.5 ACU steps; 'unpredictable/intermittent load'), parallel query, cloning (copy-on-write instant test copies), Backtrack (rewind MySQL cluster in place).

3.3 DynamoDB — the serverless workhorse

- **Data model:** tables of items (≤400 KB) with a **partition key** (hashed to spread data across partitions) and optional **sort key** (range queries within a partition). Choose a high-cardinality partition key; a 'hot' key (one popular value) throttles that partition — the classic performance question.
- **Capacity: On-demand** (pay per request, no planning — spiky/unknown traffic) vs **Provisioned** RCU/WCU with auto scaling (cheaper for steady, predictable load). Reads: eventually consistent (default, half cost) vs strongly consistent.
- **Indexes:** Global Secondary Index = query on a different partition/sort key (own capacity); Local Secondary Index = same partition key, different sort key (must be created at table creation).
- **Features: DAX** = in-memory read cache delivering MICROSECONDS with no code change (API-compatible) — 'reads still too slow' → DAX; **Streams** = ordered change log → Lambda (triggers, replication, audit); **TTL** = auto-expire items free (sessions, temp data); **Global Tables** = multi-Region active-active; **transactions** across items; PITR 35 days; export to S3 for analytics (query with Athena rather than scanning the table).

3.4 Purpose-built databases — one-liners

Service	Model	Trigger
ElastiCache Redis	In-memory, rich structures (sorted sets, pub/sub, geo), replication + persistence	Cache DB reads; sessions; LEADERBOARDS/ranking; real-time counters
ElastiCache Memcached	Simple in-memory key-value, multithreaded, no persistence/HA	Simplest possible cache, disposable
MemoryDB for Redis	Redis-compatible DURABLE primary database (multi-AZ transaction log)	'Microsecond reads AND data must survive node loss without a separate DB'
Redshift	Columnar MPP warehouse; Spectrum queries S3 in place; RA3 nodes; Serverless option	OLAP, BI, 'analyze years of data with SQL', 'petabyte warehouse'
OpenSearch Service	Search/log analytics engine + dashboards	Full-text search, log analytics (with Firehose), clickstream
DocumentDB	MongoDB-compatible document DB	Existing MongoDB app
Keyspaces	Cassandra-compatible	Existing Cassandra app
Neptune	Graph (property graph / RDF)	Social graphs, recommendations, FRAUD RINGS, knowledge graphs
Timestream	Time series	IoT sensor / telemetry / metrics over time
QLDB	Immutable, verifiable ledger	'Cryptographically verifiable history of changes' (not blockchain-decentralized)

EXAM TIP: 'Move analytics/reporting off the production RDS' has two answers depending on scale: read replica (moderate reporting) or Redshift/ETL to S3+Athena (true warehouse-scale analytics). 'Key-value + serverless + global' is DynamoDB every time.

4. Caching & Content Delivery

4.1 Where caching lives

Caches sit at four layers, and each has an exam service: (1) **edge** — CloudFront caches responses near users; (2) **API** — API Gateway stage cache; (3) **application** — ElastiCache/DAX in front of the database; (4) **database** — read replicas offload reads. The performance-tuning question usually asks which layer the bottleneck lives in: slow static content globally → (1); repeated identical API calls → (2); hot DB reads → (3); heavy but varied reads → (4).

- Caching strategies: **lazy loading** (cache-aside: check cache, miss → read DB, populate; stale data possible; only requested data cached) vs **write-through** (update cache on every write; always fresh; cache churn). TTLs bound staleness.

4.2 CloudFront in depth

- Global CDN of edge locations + Regional edge caches. **Origins:** S3 (private via Origin Access Control — replaces legacy OAI), ALB/EC2/API Gateway/any HTTP endpoint, with origin groups for failover. **Behaviors:** per path pattern (cache /static/* long, pass /api/* through), TTL controls, cache keys by header/cookie/query, compression.
- **Security:** HTTPS with ACM certs (us-east-1), WAF + Shield integration, **signed URLs/cookies** for private content (signed URL = one file; signed cookies = many files, e.g., a whole video course), geo restriction (block/allow countries), field-level encryption for sensitive form fields, Origin Access Control keeps S3 private.
- **Edge compute:** CloudFront Functions (JavaScript, sub-ms, viewer request/response only — URL rewrites, header manipulation) vs Lambda@Edge (Node/Python, more time and memory, all four hooks incl. origin request/response — auth, A/B, dynamic origin selection).
- Cost angle: data from AWS origins to CloudFront is free; edge caching cuts origin egress and load. **Invalidations** purge cached objects (or use versioned filenames).

4.3 Global Accelerator — not a CDN

Gives your app two static **anycast IPs**; users enter the AWS backbone at the nearest edge and are routed to the closest healthy endpoint (ALB/NLB/EC2/EIP) across Regions. No caching — it accelerates and stabilizes CONNECTIONS. Wins for TCP/UDP non-HTTP traffic (gaming, VoIP, IoT), for instant failover without DNS/TTL delays, and for 'we need fixed IPs for whitelisting in front of an ALB.' CloudFront = cacheable HTTP content; Global Accelerator = everything else at the network layer.

5. Hybrid Network Performance

	Site-to-Site VPN	Direct Connect (DX)
Path	Encrypted IPsec over the public internet	Dedicated private fiber to an AWS DX location
Bandwidth	~1.25 Gbps per tunnel (2 tunnels per connection; ECMP with TGW to aggregate)	Dedicated 1/10/100/400 Gbps; hosted 50 Mbps–10 Gbps
Latency	Variable (internet)	Consistent, low
Setup	Minutes–hours	Weeks to months (physical circuit)
Encryption	Built in	NONE by default — add IPsec VPN over DX, or MACsec on 10/100 Gbps dedicated
Cost	Low hourly + data	Port hours + lower per-GB egress than internet
Pick when	Fast, cheap, encrypted; backup for DX	Consistent throughput, large volumes, private, lower egress cost

- DX building blocks: **Private VIF** → one VPC (via VGW); **Transit VIF** → Transit Gateway (many VPCs); **Public VIF** → AWS public endpoints (S3, DynamoDB) privately. **DX Gateway** lets one connection reach VPCs in multiple Regions. **LAG** bundles links. Resilience: two DX connections at different locations (max) or DX + VPN failover (cost-conscious). **Accelerated VPN** routes VPN over Global Accelerator for steadier internet paths.

EXAM TIP: 'Consistent network performance for large daily transfers' → DX. 'Encrypted today' → VPN. 'DX must be encrypted' → VPN over DX. 'Cheapest DX resilience' → VPN backup. 'Hybrid DNS resolution' → Route 53 Resolver inbound/outbound endpoints.

6. Streaming, Ingestion & Analytics

6.1 Batch vs streaming

Batch collects data and processes it periodically (nightly ETL): tools = S3 + Glue + Athena/EMR/Redshift. **Streaming** processes records continuously as they arrive (clickstreams, IoT, logs): tools = Kinesis / MSK + consumers. Questions hinge on latency words: 'real time / sub-second' → Kinesis Data Streams; 'near real time / within a minute is fine / just land it in S3' → Firehose; 'hourly/daily' → batch.

6.2 The Kinesis family

	Kinesis Data Streams	Amazon Data Firehose
Role	Durable real-time stream you build producers/consumers for	Managed DELIVERY pipe: ingest → (optional Lambda transform) → destination
Latency	~200 ms (real time)	Buffered: ≥60 s or by size (near real time)
Capacity	Shards (1 MB/s in, 2 MB/s out each) or on-demand mode	Fully automatic scaling
Retention/replay	24 h default, up to 365 d; multiple consumers can re-read (enhanced fan-out = dedicated 2 MB/s per consumer)	No storage, no replay

	Kinesis Data Streams	Amazon Data Firehose
Destinations	Your code: Lambda, KCL apps, Flink	S3, Redshift, OpenSearch, Splunk, HTTP endpoints, Datadog etc.
Ordering	Per partition key within a shard	n/a
Ops	You manage shards/consumers	Zero admin

- Pattern: Data Streams feeds Firehose (real-time processing AND archival). **Managed Service for Apache Flink** runs SQL/Java stream analytics (windowed aggregations, anomaly detection) on streams. **MSK** = managed Kafka when the requirement literally says Kafka or needs its ecosystem.

6.3 Data lake analytics on S3

- **Glue:** serverless Spark ETL. **Crawlers** discover schemas into the **Data Catalog** (the metastore Athena/EMR/Redshift Spectrum all share); jobs transform and convert formats; DataBrew = visual no-code prep. 'Catalog + transform data in S3 without servers' → Glue.
- **Athena:** serverless SQL directly over S3 (Presto/Trino), pay per TB scanned. Cut cost/latency by converting to **columnar Parquet/ORC**, **compressing**, and **partitioning** (e.g., by date) so queries scan less. 'Query CloudTrail/VPC Flow/ALB logs with SQL, no infrastructure' → Athena. Federated queries reach other sources.
- **EMR:** managed Hadoop/Spark/Hive/Presto clusters — pick when you need those frameworks with cluster-level control or long custom jobs. Cost: Spot for task nodes, EMR Serverless, EMRFS to keep data in S3 (transient clusters).
- **Redshift:** the warehouse; **Spectrum** queries S3 without loading; COPY from S3 is the fast load path (Firehose can deliver straight in). **QuickSight:** serverless BI dashboards (SPICE in-memory engine). **Lake Formation:** governed data lake with row/column-level permissions across Athena/Redshift/EMR.

EXAM TIP: Real time + replay/multiple consumers → Data Streams. Simplest path to S3/Redshift/OpenSearch → Firehose. Serverless ad-hoc SQL on S3 → Athena (Parquet + partitions when cost appears). Schema discovery/ETL → Glue. BI at warehouse scale → Redshift + QuickSight.

7. Reference Architecture: Serverless Global API + Clickstream Analytics

A mobile app with users on four continents needs a low-latency API, a product catalog with millisecond reads, and real-time dashboards of user behavior plus long-term analysis.

- **API tier:** Route 53 latency routing → API Gateway (edge-optimized) → Lambda in two Regions. Cognito User Pool authorizer validates JWTs. Provisioned concurrency on the hot functions; HTTP API type to cut cost.
- **Data tier:** DynamoDB Global Table (active-active in both Regions), partition key = productId with a GSI for category browsing; DAX in front for microsecond catalog reads; TTL on session/cart items.
- **Streaming:** app events → Kinesis Data Streams (on-demand mode) → (a) Managed Flink computes 1-minute windowed metrics → OpenSearch dashboards; (b) Firehose (Parquet conversion, date partitions) → S3 data lake.
- **Batch analytics:** Glue crawler catalogs the lake; analysts query with Athena; QuickSight dashboards; heavy modelling via Redshift Spectrum without moving data. Lifecycle policies age the lake to Glacier.
- **Static assets:** S3 + CloudFront with OAC; images served via signed cookies for premium content.

Every managed choice here was picked because the requirements never asked for OS control — that's the Domain 3 reflex: reach for the highest-level service that meets the performance need.

8. Worked Practice Questions

Practice Q1. A tightly coupled HPC simulation needs the lowest possible network latency between 30 EC2 instances that exchange data constantly. What should the architect use?

✗ Spread placement group — maximizes separation (and caps at 7 per AZ); the opposite goal.

✗ Partition placement group — isolates failure domains for distributed data systems, not latency.

✓ Cluster placement group + EFA-enabled instances in one AZ — same rack proximity plus OS-bypass networking for MPI. 'Tightly coupled / low latency between nodes' = cluster.

Practice Q2. A Lambda function behind API Gateway processes uploads that take 3–4 minutes. Clients receive timeouts. Least-change fix that keeps the work serverless?

✗ Increase the API Gateway timeout — hard maximum is 29 seconds.

✗ Move the work to EC2 — abandons serverless unnecessarily.

✓ Make it asynchronous: API Gateway returns immediately (e.g., puts the job on SQS or starts a Step Functions execution); the Lambda (up to 15 min) processes in the background; clients poll or get notified. 29-second API limit vs 15-minute Lambda limit is a favorite pairing.

Practice Q3. An e-commerce catalog on DynamoDB serves 500,000 reads/second of the same few thousand hot items; latency must drop below 1 ms and no application rewrite is allowed. Solution?

✗ Increase provisioned RCUs — throughput yes, sub-ms latency no; and hot keys still throttle.

✗ ElastiCache Redis — works but requires cache-aside code changes.

✓ DynamoDB Accelerator (DAX) — API-compatible in-memory cache delivering microsecond reads with only an endpoint change. 'DynamoDB + faster reads + no code change' → DAX.

Practice Q4. A company needs to run 2,000 nightly image-processing jobs, each 20–90 minutes, at minimal cost with no servers to manage between runs. Best fit?

✗ Lambda — 15-minute limit.

✗ Always-on ECS EC2 cluster — idle cost all day.

✓ AWS Batch with a Fargate (or Spot EC2) compute environment — managed job queues, scales to zero, jobs of any length, Spot for deep discounts. 'Many long batch jobs, scale to zero' = Batch.

Practice Q5. Logs are written to S3 in JSON. Analysts need occasional ad-hoc SQL over 2 years of logs with no infrastructure to manage, and query costs are rising. What should the architect do?

✗ Load everything into Redshift — warehouse running 24/7 for occasional queries; high fixed cost.

✗ EMR cluster — infrastructure to manage, contrary to the requirement.

✓ Athena over S3, with a Glue job converting JSON to partitioned, compressed Parquet — serverless, and columnar + partitions slash the TBs scanned per query. Athena cost = bytes scanned; shrink the scan.

Practice Q6. A multiplayer game's UDP traffic must reach the nearest healthy Region with static IP addresses and failover within seconds. Which service?

✗ CloudFront — HTTP(S) content only; caches; no UDP.

✗ Route 53 latency routing — works but failover waits on DNS TTLs and IPs aren't static.

✓ AWS Global Accelerator with endpoint groups in each Region — static anycast IPs, backbone routing, health-based failover in seconds, TCP/UDP. 'UDP / static IP / instant failover' = Global Accelerator.

Practice Q7. Sensor data (thousands of records/second) must be analyzed in real time for anomalies AND retained in S3 for later queries, with minimal operational effort. Best design?

✗ Firehose only — delivers to S3, but 60-second buffering isn't real-time analysis and there's no processing hook beyond simple transforms.

✗ EC2 consumers reading directly — operational burden.

✓ Kinesis Data Streams → Managed Service for Apache Flink for real-time anomaly detection, plus a Firehose consumer delivering to S3. Streams for real time + fan-out, Firehose for the archive path.

Practice Q8. A company with a 10 Gbps Direct Connect link must encrypt all traffic to AWS for compliance, at the lowest additional cost and complexity. Options?

✗ Replace DX with VPN — loses the dedicated bandwidth/consistency they paid for.

✗ Rely on application TLS only — not 'all traffic'.

✓ Run an IPsec Site-to-Site VPN over the Direct Connect connection (or enable MACsec if the link is dedicated 10/100 Gbps and both ends support it). DX is unencrypted by default; VPN-over-DX is the standard fix.

Self-check before Module 4

- Walk the compute spectrum EC2 → Lambda and name what pushes you left or right.
- Lambda: timeout, memory range, API Gateway integration limit, what to do for RDS connections.
- Relational vs DynamoDB vs Redshift vs Redis — one trigger phrase each.
- CloudFront vs Global Accelerator: which caches, which handles UDP, which gives static IPs?
- Data Streams vs Firehose: latency, replay, who manages consumers?
- VPN vs Direct Connect: setup time, encryption default, and how you encrypt DX.